



3-DAY COURSE

GitHub Administration

Day 1: Foundations, Collaboration & Repository Management

Welcome & Course Objectives



Navigate and configure the GitHub environment and your account/profile



Manage repositories: create, clone, archive, and handle files (including large files)



Use GitHub's project tools — issues, pull requests, project boards



Understand security fundamentals (previewed today, deep dive on Day 3)



Collaborate effectively — teams, permissions, and roles



Use advanced features: GitHub Pages, wikis, and repository insights

Day 1 Agenda



MODULE 1

Getting Started with
GitHub



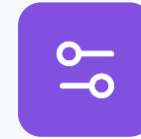
MODULE 2

Communication &
Collaboration Tools



MODULE 3

Repository
Management



MODULE 4

Advanced Repository
Operations

MODULE 01

Getting Started with GitHub

Accounts, profiles, exploring projects, and writing on GitHub

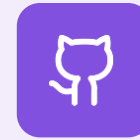
What Is GitHub? Platform Overview



Git

- A distributed version control system (a program), created by Linus Torvalds in 2005
- Runs entirely on your local machine — no internet connection required
- Tracks every change to every file as a chain of commits (snapshots)
- Command-line first, though many graphical tools exist

VS



GitHub

- A cloud-hosting company (a product/service), built around Git, owned by Microsoft
- Adds a web UI, remote storage, and collaboration layers on top of Git
- Adds issues, pull requests, project boards, wikis, Pages, and Actions
- Used for open-source projects, private company codebases, docs, and more

Exploring Projects on GitHub



GitHub hosts tens of millions of public repositories across every domain and language



Evaluate a repo's health before using or contributing: stars, forks, recent commit activity, open issues



Check for a README, LICENSE, and CONTRIBUTING file as signs of a well-maintained project



Look at the Insights tab for contributor activity and commit frequency graphs (covered in depth in Module 4)



Watch for red flags: no recent commits, unanswered issues piling up, no license

Using GitHub Explore to Discover Projects



GitHub Explore (github.com/explore) surfaces trending and recommended repositories



Trending page filters by language and time range (daily/weekly/monthly)



Topics pages group repositories by subject — e.g., machine-learning, react, devops



Personalized recommendations improve as your account follows people and stars repos



Useful for discovering tools, learning by reading real-world code, and finding projects to contribute to

Setting Up Your GitHub User Account



Sign up at github.com with a professional-appropriate username and email



Choose a plan — Free covers everything needed for this course



Verify your email address to unlock full account functionality



Account settings hub controls profile, security, notifications, billing, and integrations



One account can belong to many organizations later on (covered Day 2, Module 6)

Authenticating to GitHub



Password + 2FA

Standard web login, hardened with a required second factor (authenticator app, security key, or SMS)



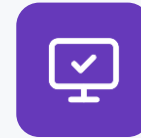
Personal Access Token

A revocable token that stands in for your password for the CLI, APIs, and third-party tools



SSH Keys

A public/private key pair enabling passwordless, secure Git operations from the command line



CLI & Desktop Apps

GitHub CLI (gh) and GitHub Desktop handle their own authenticated login flow for you

Setting Up Your GitHub Profile



Your profile page (github.com/username) is your public developer identity



Add a profile photo, display name, bio, location, and company/pronouns (optional)



Pin up to 6 repositories to highlight your best or most relevant work



A special README repository ([username/username](https://github.com/username/username)) renders directly on your profile page



Profile visibility settings control what activity is shown publicly

Writing an Effective Bio & Showcasing Your Work



Keep your bio concise: role, focus area, and what you're currently working on or learning



Pin repositories that best represent your skills, not just your most recent work



Use the profile README to add a short intro, tech stack badges, or links to a portfolio



Link out to a personal site, LinkedIn, or portfolio where relevant



Keep it current — a stale profile undersells active contributors

Writing on GitHub: Markdown Basics

```
README.md

# Project Title

A short, italic description.

- First item

- Second item

`inline code` and a link:

[GitHub] (https://github.com)
```

PREVIEW

Project Title

A short, *italic* description.

- First item
- Second item

inline code and a link: GitHub

Documenting Projects: Writing READMEs



Title & Badges

Project name plus build/version/license badges for a professional first look



Description

One or two sentences: what the project does and who it's for



Installation

Exact commands needed to get the project running locally



Usage

Real examples showing common commands or API calls in action



Contributing

How outside contributors can propose changes or file issues



License

The legal terms under which the code can be used, copied, or modified

Module 1 Recap

- ✓ GitHub = hosted collaboration layer on top of Git
- ✓ Explore and evaluate repositories using activity signals
- ✓ Account set up, secured with 2FA, and authenticated (PAT/SSH)
- ✓ Profile personalized with bio, pinned repos, and profile README
- ✓ Markdown mastered for READMEs, issues, and PRs

MODULE 02

Communication & Collaboration Tools

Notifications, Gists, wikis, and GitHub Pages

Receiving Notifications About Activity on GitHub



GitHub notifies you on issues, PRs, mentions, and repos you watch



The notifications inbox (github.com/notifications) centralizes everything in one place



Notifications can be delivered via the web inbox, email, or both



Watching a repo (vs. just starring it) subscribes you to its activity



Unread/read/done states help manage a busy inbox

Setting Notification Preferences



Configure globally under Settings > Notifications



Choose default behavior for participating vs. watching activity separately



Set custom routing rules to send certain org/repo notifications to specific email addresses



Enable/disable notifications for GitHub Actions, security alerts, and Dependabot separately



Mobile app and web push notifications can be tuned independently

Managing Email & Web Notifications



Email notifications arrive from a distinct notifications@github.com-style address — filter/label accordingly



Web notifications inbox supports filtering by reason: assigned, mentioned, review requested, etc.



Mark as read, save for later, or unsubscribe directly from a thread



Use the inbox's query syntax to filter by repo, org, or notification reason



Digest emails can reduce volume for less time-sensitive updates

Working with Gists



Gists (gist.github.com) are lightweight, shareable code snippets — each Gist is its own mini Git repo



Public Gists are discoverable and can be embedded in blogs and web pages



Secret Gists are unlisted (not truly private — don't use for sensitive data)



Gists support multiple files, revision history, comments, and forking



Great for sharing quick examples, config snippets, or one-off scripts without creating a full repository

Using the GitHub Wiki



Every repository can have an attached Wiki for longer-form documentation



Wikis are themselves Git repositories — can be cloned and edited locally



Support Markdown (and other formats) with a built-in page editor and sidebar navigation



Useful for architecture docs, onboarding guides, FAQs, and meeting notes



Access controlled separately from the main repository — can restrict wiki editing to collaborators

Wiki Best Practices



Start with a clear Home page that links to all other pages — treat it like a table of contents



Keep a consistent page naming and heading structure across the wiki



Assign an owner or review cadence so wiki content doesn't go stale



Prefer the README for 'how to run this project' and the wiki for deeper reference material



Decide deliberately: in-repo docs/ folder vs. wiki, based on your team's review workflow

GitHub Pages Basics



GitHub Pages turns a repository (or branch/folder) into a hosted static website — free hosting



Three site types: personal/organization site, and project sites



Supports plain HTML/CSS/JS or static site generators like Jekyll (built-in support)



Sites are served from a gh-pages branch, a /docs folder, or GitHub Actions-based deployment



Custom domains are supported, with free HTTPS via GitHub's certificate management

Hosting Personal, Organization & Project Pages



Personal / Org Site

- Repo must be named exactly username.github.io (or orgname.github.io)
- Only one per account — represents you or your org broadly
- Deployed automatically from the default branch
- Lives at the root URL: <https://username.github.io>

VS



Project Site

- Any repository name works — no special naming required
- Unlimited — one possible per repository you own
- Published from a chosen branch, folder, or Actions workflow
- Lives at a sub-path: <https://username.github.io/repo-name>

Customizing GitHub Pages



Choose or customize a Jekyll theme via `_config.yml` for quick styling



Add a custom domain in Settings > Pages, plus a CNAME file in the repo



Enforce HTTPS once a custom domain's certificate is provisioned



Use a docs/ folder or dedicated branch to keep source and published site organized



Combine with GitHub Actions for build steps beyond Jekyll (React, Vue, static generators, etc.)

Module 2 Recap

- ✓ Notifications tuned for signal over noise (web + email)
- ✓ Gists used for quick, shareable code snippets
- ✓ Wikis set up with clear structure for longer-form docs
- ✓ GitHub Pages basics: personal/org vs. project sites
- ✓ Pages customized with themes, folders, or custom domains

Repository Management

Creating, cloning, files, large files, and remotes

Creating, Cloning & Archiving Repositories



Create a new repository from github.com/new — choose visibility, README, .gitignore, and license



Clone brings a full copy of a repo (and its history) to your local machine via HTTPS or SSH



Fork creates your own copy of someone else's repository under your account



Archiving makes a repo read-only — ideal for completed or deprecated projects



Archived repos can be unarchived later if you need to reopen them for changes

Steps to Create and Clone a Repository

- 1 Click New on github.com or the + menu in the top navigation
- 2 Name the repository and choose Public or Private visibility
- 3 Optionally initialize with a README, .gitignore, and license template
- 4 Click Create repository
- 5 Copy the HTTPS or SSH clone URL from the green Code button
- 6 Run `git clone <url>` in your terminal to pull it down locally

Archiving and Restoring Repositories



Archive from Settings > General > Danger Zone > Archive this repository



Archived repos remain visible and cloneable but become fully read-only



Issues, PRs, and pushes are disabled until the repo is unarchived



Unarchive from the same Danger Zone panel when work needs to resume



Use archiving over deletion when history and context should be preserved

Managing Files in a Repository



Add files via the web UI (drag-and-drop or 'Add file'), Git CLI, or GitHub Desktop



Edit files directly in the browser for quick fixes — creates a commit automatically



Delete files through the UI or with `git rm` locally, then commit and push



Every file change is tracked in commit history with an author, timestamp, and message

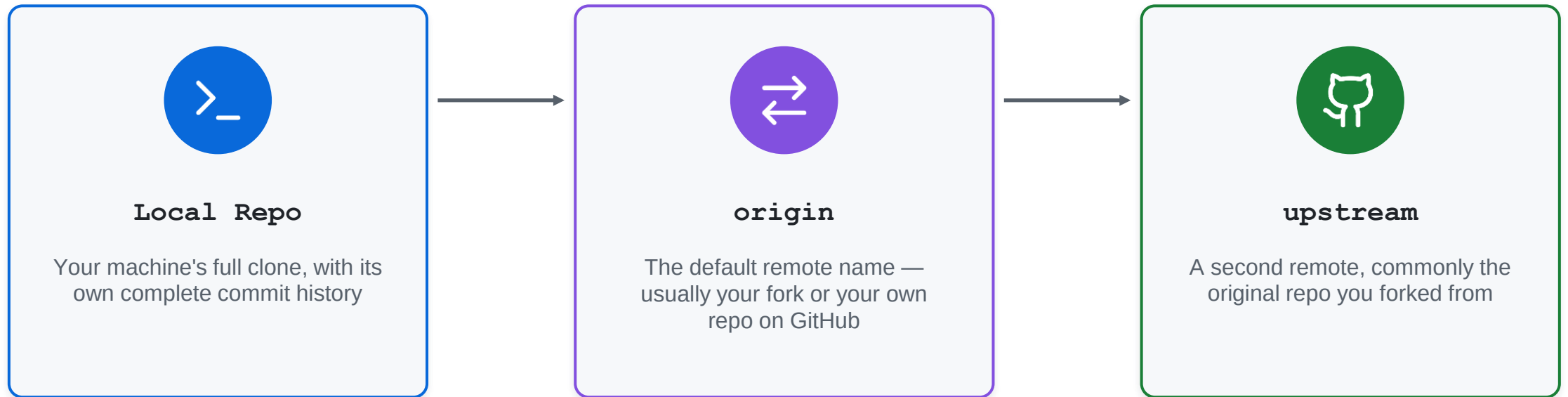


Use meaningful commit messages — they become the permanent record of why a change was made

Non-Code Files & Managing Large Files with Git LFS



Managing Remotes



Fetching and Pushing Changes to Remotes



Module 3 Recap

- ✓ Repositories created, cloned, and archived confidently
- ✓ Files added, edited, and deleted with clear commit history
- ✓ Large/non-code files handled properly with Git LFS
- ✓ Remotes configured, listed, added, and removed
- ✓ Fetch vs. pull vs. push distinguished clearly

Advanced Repository Operations

Administration, branches/tags, importing, and insights

Administering a Repository



Settings > General controls visibility (public/private/internal), features, and merge options



Danger Zone handles high-impact actions: transfer ownership, archive, delete



Collaboration settings define who can push, review, or administer the repo

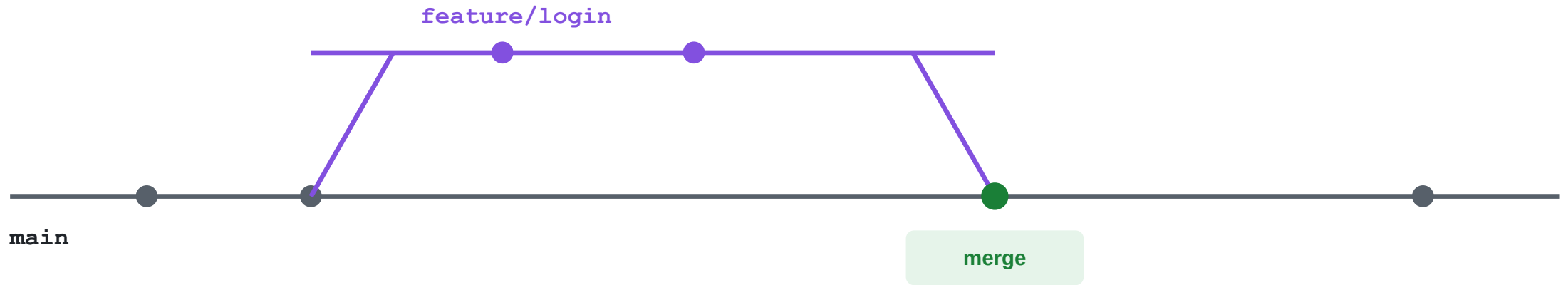


Branch protection rules enforce review requirements and status checks before merging



Repository-level settings are distinct from organization- and account-level settings

Managing Branches



Create a branch

`git checkout -b feature/login` — starts as an exact copy of main's current state



Commit independently

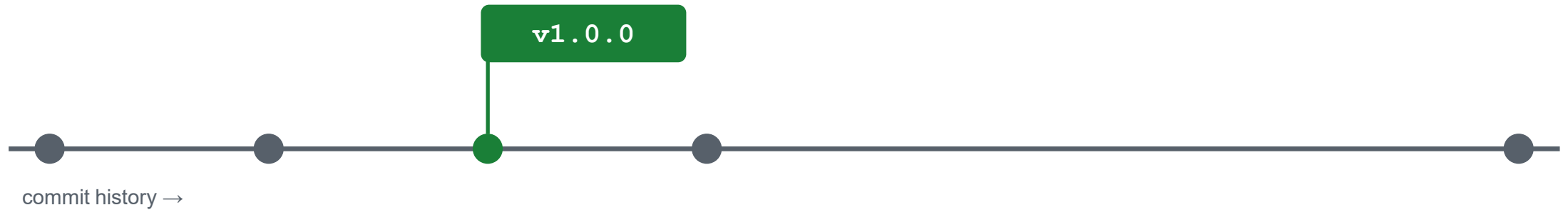
New commits land only on the branch; main is untouched until merge



Merge back

A pull request merges the branch's changes into main, then the branch can be deleted

Managing Tags



Release v1.0.0

A tag can be promoted into a full GitHub Release: formatted notes, a diff since the last release, and downloadable binary assets attached directly to this version.



Branch = moving pointer

Advances automatically with every new commit



Tag = fixed marker

Permanently references one exact commit, forever

Importing Your Projects to GitHub



GitHub Importer (github.com/new/import) migrates repos from Git, Subversion, Mercurial, and TFS



Provide the source repository URL and, if needed, source credentials



Importer preserves commit history where the source system supports it



For existing local Git repos, simply add GitHub as a remote and push



Large or complex imports may require the command-line `git clone --mirror` approach

Migrating from Other Version Control Systems



Subversion (SVN)

GitHub Importer supports direct SVN-to-Git migration, converting revisions to commits



Mercurial (Hg)

Convert using tools like hg-git or fast-export before importing into GitHub



TFVC (Azure DevOps)

Team Foundation Version Control is supported via the Importer for Azure DevOps sources

Visualizing Repository Data with Graphs



Pulse

Recent activity summary
— merged PRs, opened
issues, active
contributors



Contributors

Commits over time per
contributor, ranked by
activity



Commits

Overall commit
frequency across the
project's whole timeline



Traffic

Views and clones over
the last 14 days (visible
to admins only)

Network, Contributors & Commit Activity Graphs



Network Graph

Visualizes every branch and merge across the repo as a timeline



Contributors Graph

Ranks contributors by commits, plus additions and deletions



Commit Activity

Commits grouped by week over the trailing year



Code Frequency

Additions vs. deletions over time, visualized as two trend lines

Using Insights for Project Management



Spot stalled projects early: declining commit frequency or a growing backlog of open issues



Identify review bottlenecks by watching pull request cycle time trends



Recognize contributor burnout risk when activity concentrates on very few people



Use traffic data to understand which repos/docs get the most external attention



Combine graphs with project boards (Day 2, Module 6) for a fuller project-health picture

Module 4 Recap

- ✓ Repository settings, Danger Zone, and branch protection understood
- ✓ Branches created/deleted; tags created and pushed for releases
- ✓ Projects imported from other VCS platforms via the Importer
- ✓ Insights graphs (Pulse, Contributors, Commits, Network, Traffic) explored
- ✓ Graphs connected to real project-management decisions

Day 1 Key Takeaways

- ✓ GitHub extends Git with collaboration, hosting, and automation tools
- ✓ A secure, well-configured account and profile is the foundation for everything else
- ✓ Communication tools (notifications, Gists, wikis, Pages) support teams beyond just code
- ✓ Solid repository management practices (files, LFS, remotes) prevent painful cleanup later
- ✓ Advanced repo operations (branches, tags, insights) set the stage for Day 2's collaboration workflows

GitHub Terminology Cheat Sheet



Repository (repo)

A project's folder of files plus its full version history



Clone

A local copy of a remote repository, including its history



Fork

Your own copy of someone else's repository under your account



Commit

A saved snapshot of changes with an author and message



Branch

An independent line of development within a repository



Remote

A named reference to a repository hosted elsewhere (e.g., origin)



Tag

A fixed marker on a specific commit, often used for releases



PAT

Personal Access Token — used to authenticate CLI/API requests

Common Pitfalls & Best Practices



Enable 2FA today if you haven't already — don't wait for a security incident



Write commit messages for your future self and your teammates, not just today's you



Prefer archiving over deleting when a repo's history might have future value



Use Git LFS from the start for large binaries — retrofitting history later is painful



Keep READMEs and wikis current; stale docs erode trust faster than no docs at all

Day 1 Lab Overview

1

Create & Configure Your GitHub Account and Profile

2

Explore & Evaluate Repositories

3

Write a README with Markdown

4

Configure Notification Preferences

5

Create & Share a Gist

6

Set Up a Repository Wiki

7

Publish a Site with GitHub Pages

8

Create, Clone & Archive a Repository

9

Manage Files, Git LFS & Remotes

10

Branches, Tags & Repository Administration

Resources & Further Reading



GitHub Docs — docs.github.com (official, authoritative reference for every feature covered today)



GitHub Skills — skills.github.com (free, interactive, hands-on courses)



Git documentation — git-scm.com/doc (for deeper Git fundamentals)



GitHub Changelog — github.blog/changelog (track new features as they ship)



This course's lab guide — keep it handy as a reference during and after class

COMING UP

Looking Ahead: Day 2 Preview



Module 5: Search and Integration — GitHub search, webhooks, and GitHub Actions



Module 6: Collaboration and Team Management — organizations, teams, issues, and pull requests



Module 7: Practical Exercises and Case Studies — applied hands-on practice



Bring your Day 1 laptop setup — Day 2 builds directly on today's account and repositories



3-DAY COURSE

GitHub Administration

Day 2: Search, Automation & Team Collaboration

Welcome & Day 2 Objectives



Use GitHub's search syntax to find code, issues, and pull requests across repositories



Understand webhooks and how external systems react to repository events



Build and secure a GitHub Actions workflow, from trigger to job to secret



Create and structure a GitHub organization with teams and permissions



Run a complete issue and pull request workflow, including required review



Apply everything learned so far to realistic, scenario-based case studies

Day 2 Agenda



MODULE 5

Search and
Integration



MODULE 6

Collaboration and
Team Management



MODULE 7

Practical Exercises
and Case Studies

MODULE 05

Search and Integration

GitHub search, webhooks, and GitHub Actions

05

GitHub Search Fundamentals



The global search bar at the top of every page searches across all of GitHub you have access to



Search scopes include code, issues, pull requests, repositories, commits, and users



Results can be filtered by language, visibility, organization, and dozens of other facets



Sort results by best match, most stars, most forks, or most recently updated



Search results respect your permissions — private repositories only appear if you have access

Search Qualifiers and Syntax

```
search query

# find open bugs in one org

org:acme-corp is:issue is:open

label:bug

# find code by language

filename:config.yml language:yaml

# find recently merged PRs

is:pr is:merged
```

PREVIEW

What each qualifier does

- org: scopes results to every repository in an organization
- is:issue / is:pr narrows to one result type
- is:open / is:merged filters by current state
- label:bug matches a specific issue label
- language: filters code search by file language
- merged:>DATE filters by a date range

Searching Code Across Repositories



Code search indexes the default branch of every repository you have access to



Search for exact strings, function names, or configuration values across an entire organization



Combine language:, path:, and extension: qualifiers to narrow results precisely



A common administrative use: searching for accidentally committed secrets or hardcoded credentials



Results link directly to the exact line and file, with surrounding context shown inline

Saved Searches and Search Scope



Frequently run queries can be bookmarked for one-click reuse



Scope a search to a single repository, an organization, or all of GitHub



Organization-wide search is available to any member, not just owners



Advanced search pages provide a form-based alternative to typing qualifiers by hand



Search indexes update continuously, typically reflecting new commits within moments

What Are Webhooks?



A webhook sends an HTTP POST request to a URL you specify whenever a chosen event occurs



Events include pushes, new issues, pull request activity, releases, and dozens more



Common destinations: a CI server, a Slack integration, or a custom internal service



Webhooks can be configured at the repository, organization, or GitHub App level



Each delivery includes a JSON payload describing exactly what happened

Configuring a Webhook

- 1 Open the repository or organization's Settings, then click Webhooks
- 2 Click Add webhook
- 3 Enter the Payload URL — the endpoint that will receive the POST request
- 4 Set the Content type to application/json
- 5 Enter a Secret to sign each payload for verification
- 6 Choose which events trigger the webhook, then click Add webhook

Webhook Payloads and Securing Deliveries



Every payload includes an event type, the repository, the actor, and event-specific details



A shared secret lets the receiving system verify a payload's signature before trusting it



GitHub retains a full delivery history, including headers, payload, and response for each attempt



Failed deliveries can be manually redelivered directly from the Recent Deliveries panel



Always require HTTPS for the payload URL — GitHub will reject plain HTTP by default

Introduction to GitHub Actions



GitHub Actions runs automated workflows directly on GitHub's own infrastructure



Workflows are defined in YAML files stored in a repository's `.github/workflows/` folder



A workflow is triggered by an event, then runs one or more jobs made of individual steps



The Actions Marketplace offers thousands of pre-built steps for common tasks



Actions usage is metered by the minute, with different quotas per plan tier

Anatomy of a Workflow File



name:

A human-readable name shown in the Actions tab



on:

The event(s) that trigger this workflow — push, pull_request, schedule, etc.



jobs:

One or more independent units of work, each running on its own runner



steps:

The ordered sequence of actions or shell commands within a job



secrets:

Encrypted values referenced inside steps without exposing them in logs

Workflow Triggers (Events)



push

Runs whenever commits are pushed, optionally filtered to specific branches or paths



pull_request

Runs when a PR is opened, updated, or synchronized — ideal for CI checks before merge



schedule

Runs on a cron-style schedule, independent of any repository activity



workflow_dispatch

Runs on demand, triggered manually from the Actions tab or the API

Jobs, Steps, and the Actions Marketplace



Jobs run in parallel by default; use `needs:` to make one job wait for another



Each job runs on a fresh virtual machine (runner) with a chosen operating system



Marketplace actions are referenced as `owner/action-name@version`, e.g. `actions/checkout@v4`



Steps can also run raw shell commands directly with `run:`, without any external action



Pin actions to a specific version or commit SHA to avoid unexpected behavior changes

Reusable Workflows and Composite Actions



A reusable workflow lets one workflow file call another, avoiding duplicated YAML



Composite actions bundle multiple steps into a single custom action your team can reuse



Both can live in the same repository or a separate, shared repository across your organization



Inputs and outputs let a reusable workflow or action behave differently per caller



Standardizing on shared workflows keeps CI/CD consistent across dozens of repositories

Securing Actions with Secrets



Secrets store sensitive values (API keys, tokens, passwords) encrypted at rest



Configurable at the repository, environment, or organization level



Secret values are automatically redacted from workflow logs, even if accidentally printed



Organization-level secrets can be scoped to specific repositories, not shared with all of them



Secrets are never exposed to workflows triggered from a forked repository's pull request by default

GitHub-Hosted vs. Self-Hosted Runners



GitHub-Hosted

- Fresh virtual machine provisioned automatically for every job
- Ubuntu, Windows, and macOS images maintained and patched by GitHub
- Usage billed by the minute against your plan's included quota
- Zero infrastructure to maintain — the simplest default choice

VS



Self-Hosted

- Your own machine or server registered as a runner for your repository or org
- Full control over hardware, pre-installed software, and network access
- No per-minute Actions billing, but you own the infrastructure cost
- Common for large builds, specialized hardware, or internal network access

Monitoring Actions Usage and Minutes



Usage is tracked under Settings > Billing and plans, broken down by repository



Each plan tier includes a monthly quota of free minutes before additional usage is billed



Minutes are multiplied by OS — Windows and macOS runners cost more per minute than Linux



Spending limits can be set to cap usage before it results in unexpected charges



The Actions tab itself shows recent workflow run history, duration, and success/failure status

Module 5 Recap

- ✓ GitHub search syntax mastered for code, issues, and pull requests across an organization
- ✓ Webhooks configured, signed with a secret, and understood as GitHub-to-external notifications
- ✓ GitHub Actions workflow structure understood: triggers, jobs, steps, and the Marketplace
- ✓ Secrets scoped correctly at the repository, environment, and organization level
- ✓ GitHub-hosted vs. self-hosted runners and Actions usage monitoring both covered

MODULE 06

Collaboration and Team Management

Organizations, teams, issues, and pull requests

What Is a GitHub Organization?



A shared account that owns repositories, teams, and members — distinct from any one person



Repositories can belong to an organization instead of an individual account



Membership is managed centrally, independent of any single repository



Organizations unlock security and compliance features not available on personal accounts



Billing, plan tier, and policy are configured once, at the organization level

Organization Roles: Owner, Member, Billing Manager



Owner

Full administrative control — settings, billing, member management, and repository access



Member

Default role for anyone added to the organization; repository access is granted separately



Billing Manager

Manages billing and payment settings only, without access to code or member management

Creating and Configuring an Organization

- 1 From the + icon, select New organization and choose a plan
- 2 Choose a unique organization name — this becomes part of every repository's URL
- 3 Add your organization's contact email and confirm your own account as the owner
- 4 Invite initial members by username or email, assigning an initial role to each
- 5 Review and set the default repository permission level for new members
- 6 Open Settings to configure profile details, security defaults, and repository creation policy

Building Teams and Nested Teams



Organization

The top-level account that owns every repository and every team



Parent Team

A broad team, e.g. 'Engineering', that can be granted access across many repositories



Child Team

A nested team, e.g. 'Backend', inheriting the parent's visibility and mentions



Team Members

Individual members added to a team, gaining whatever access that team has been granted



Repository Access

Permission level (Read, Write, Maintain, Admin) granted to a team on a specific repo

Assigning Team Permissions to Repositories



From a team's page, click Repositories, then Add repository



Choose a permission level: Read, Triage, Write, Maintain, or Admin



Changing a team's permission on a repository instantly updates access for every member



A repository can be granted to multiple teams, each with a different permission level



Review a repository's full access picture under its own Settings > Collaborators and teams

Organization Security Settings: SSO and Base Permissions



SAML single sign-on can be enforced, requiring members to authenticate through your identity provider



Two-factor authentication can be required for every member of the organization



Base permissions define the default access level applied to every member automatically



IP allow lists can restrict access to requests coming from approved network ranges



Third-party application access can be restricted or require explicit organization approval

Managing Member Access and Pending Invitations



Invite members from People > Invite member, by username or email



Assign an initial role (Member or Owner) and, optionally, initial team memberships during invitation



Invitations remain pending until accepted — track status from the People tab



Remove a member entirely, or adjust their role, at any time from the same panel



Removing a member immediately revokes all their team-based repository access

Auditing Organization Activity with the Audit Log



The audit log records security-relevant events across the entire organization



Filter by actor, action type, or date range using the same qualifier syntax as search



Tracked events include permission changes, repository visibility changes, and member removals



Log data can be exported for compliance reporting or long-term retention



Retention length and export options vary by plan tier

Creating and Labeling Issues



Issues track bugs, feature requests, and any discrete unit of work



Labels categorize issues for filtering — bug, enhancement, and custom labels are common



Assignees indicate who is responsible for resolving an issue



Issues can reference and automatically link to related pull requests and other issues



Closing an issue can be automated by referencing it with a closing keyword in a PR

Milestones and Issue Templates



Milestones

Group related issues and PRs under a shared goal, often tied to a release or deadline



Bug Report Template

A structured form ensuring every bug report includes reproduction steps



Feature Request Template

A structured form for proposing new functionality with context and rationale

The Pull Request Review Workflow



Required Reviewers and Merge Strategies



Merge Commit

Preserves every individual commit from the branch, plus a new merge commit



Squash and Merge

Combines all of a branch's commits into a single, clean commit on the base branch



Rebase and Merge

Replays the branch's commits individually onto the base branch, without a merge commit

Planning Work with Projects (Boards)



Projects provide a visual board for planning work across one or many repositories



Issues and pull requests can be added as cards and organized into custom columns



Views can be filtered and grouped by assignee, label, milestone, or any custom field



Custom fields (status, priority, iteration) extend a board beyond GitHub's built-in issue fields



Organization-level projects can span every repository a team works across

Module 6 Recap

- ✓ Organizations created and configured, distinct from personal accounts
- ✓ Roles (Owner, Member, Billing Manager) and team-based permissions understood
- ✓ Organization security settings, member management, and the audit log covered
- ✓ Issues, labels, milestones, and templates used to organize real work
- ✓ The full pull request review-and-merge workflow practiced end to end

MODULE 07

Practical Exercises and Case Studies

Applying two days of administration skills to realistic scenarios

Case Study Overview: Applying Two Days of Skills



Four scenarios, each drawing on a different combination of Day 1 and Day 2 material



Work through each case study in small groups before comparing approaches as a class



There is rarely one single correct answer — the goal is a defensible, well-reasoned approach



Each case study maps directly to one of today's final labs for hands-on practice

Case Study 1: Structuring a New Organization



Scenario: a 40-person engineering department is migrating from personal accounts to a single organization



Question: how many teams, and how should they be nested?



Question: what should the default base permission level be for new members?



Question: should SSO and required 2FA be enforced from day one, or phased in?



Discuss trade-offs in small groups, then compare team structures as a class

Case Study 2: Standing Up CI with Actions



Scenario: a repository has no automated testing — every bug is caught manually, after merge



Question: which trigger should run the test suite — push, pull_request, or both?



Question: GitHub-hosted or self-hosted runners, given the team's build takes 25 minutes?



Question: what secrets, if any, does the test suite legitimately need?



Question: should the required check block merging, or just report status?

Case Study 3: Onboarding a New Engineer



Scenario: a new hire starts Monday and needs working access by 9am, not by end of week



Question: what's the correct order of operations — account, then org, then team?



Question: which existing team(s) should they join on day one versus later?



Question: how do you confirm their access actually works before their first standup?



Question: what should be documented so the next new hire's onboarding is faster?

Case Study 4: Preparing for a Compliance Access Review



Scenario: an external auditor has requested a full access review across the organization in 48 hours



Question: where do you find a complete record of who has access to what, and since when?



Question: how do you identify accounts with access they no longer need?



Question: what evidence can you export, and what has to be documented manually?



Question: what would you change about today's access model to make the next audit faster?

Discussion: Common Governance Pitfalls



Broad, individual access grants that never get revisited or revoked



A single owner account, with no backup, holding critical organization access



Unpinned Marketplace actions in CI, trusting an external maintainer's every future update



Secrets scoped more broadly than the workflow that actually needs them



No documented process for onboarding or offboarding, repeated inconsistently each time

Building a GitHub Administration Checklist



Access & Identity

2FA enforced, SSO configured, teams reflect actual current access needs



Repository Hygiene

Branch protection, required reviews, and consistent merge strategy across repos



Automation & Integration

Workflows use pinned actions and correctly scoped, minimally-privileged secrets



Documentation

Onboarding, offboarding, and access-request processes written down, not tribal knowledge



Compliance & Auditing

Audit log reviewed on a regular cadence, not only when an external audit demands it

Module 7 Recap

- ✓ Four case studies worked through, each combining Day 1 and Day 2 material
- ✓ Organization structure, CI setup, onboarding, and compliance auditing all practiced as scenarios
- ✓ Common governance pitfalls identified, each tied back to a specific earlier concept
- ✓ A five-category administration checklist built as a lasting reference
- ✓ Judgment and trade-off reasoning emphasized over any single correct configuration

Day 2 Key Takeaways

- ✓ Search and webhooks connect GitHub to information and systems beyond a single repository
- ✓ GitHub Actions automates work directly on GitHub's infrastructure, secured with scoped secrets
- ✓ Organizations and teams are the scalable pattern for managing access across many people and repos
- ✓ The full issue and pull request lifecycle ties planning directly to code review and merge
- ✓ Real administration is about combining these tools with judgment, not applying any one in isolation

Day 2 Terminology Cheat Sheet



Organization

A shared account that owns repositories, teams, and members



Team

A named group of members granted shared repository access



Webhook

An HTTP notification GitHub sends when a chosen event occurs



Workflow

A YAML-defined automation triggered by an event, made of jobs and steps



Runner

The machine (GitHub-hosted or self-hosted) that executes a workflow's jobs



Secret

An encrypted value a workflow can use without exposing it in logs



Milestone

A shared goal grouping related issues and pull requests



Audit Log

A record of security-relevant events across an organization

Common Pitfalls & Best Practices



Maintain at least two organization owners — never a single point of failure



Pin Marketplace actions to a specific version or commit SHA, not a floating tag



Scope every secret to only the workflows and repositories that genuinely need it



Grant access through teams, not individual collaborators, wherever possible



Review the audit log on a regular cadence — don't wait for an external audit to look

Day 2 Lab Overview

1

Mastering GitHub Search Syntax and Saved Searches

2

Creating and Testing a Webhook

3

Building Your First GitHub Actions Workflow

4

Securing a Workflow with Encrypted Secrets

5

Creating and Configuring a GitHub Organization

6

Building Teams and Assigning Repository Permissions

7

Configuring Organization Security Settings and the Audit Log

8

Running a Full Pull Request Review Workflow

9

Planning Work with Issues, Milestones, and a Project Board

10

Case Study — Onboarding a New Engineering Team

Resources & Further Reading



GitHub Docs — docs.github.com/actions and docs.github.com/organizations



GitHub Actions Marketplace — github.com/marketplace/actions



GitHub Search Syntax Reference — docs.github.com, under Searching on GitHub



GitHub Skills — skills.github.com, including dedicated Actions and organization courses



This course's Day 2 lab guide — keep it handy as a reference during and after class

COMING UP

Looking Ahead: Day 3 Preview



Module 8: Security — vulnerability alerts, secret scanning, CodeQL, and Dependabot



Capstone Group Project: applying all three days to a realistic organization setup



Bring your Day 2 organization and repositories — Day 3 builds directly on both



3-DAY COURSE

GitHub Administration

Day 3: Security, Compliance & the Capstone Project

Welcome & Day 3 Objectives



Understand the dependency graph and how Dependabot finds and fixes vulnerable dependencies



Configure secret scanning and push protection to catch leaked credentials automatically



Set up code scanning with CodeQL to catch vulnerabilities directly in source code



Publish security advisories and write a security policy for responsible disclosure



Enable and manage security features at scale across an entire organization



Apply all three days of material to a realistic capstone organization setup

Day 3 Agenda



MODULE 8

Security: Dependabot,
Secret Scanning,
CodeQL & Advisories



CAPSTONE

Capstone Group
Project



WRAP-UP

Course Close-Out &
Next Steps

MODULE 08

Security

Dependabot, secret scanning, code scanning, and security advisories

08

GitHub's Security Feature Set Overview



Dependency graph and Dependabot: find and fix vulnerable or outdated dependencies



Secret scanning: detect credentials and tokens committed to a repository



Code scanning (CodeQL): find vulnerable patterns directly in source code



Security advisories: coordinate and disclose vulnerabilities responsibly



Security overview: an aggregated, organization-wide view of all of the above

The Dependency Graph



GitHub automatically parses manifest files (package.json, requirements.txt, go.mod, etc.)



Builds a complete graph of direct and transitive dependencies for a repository



Cross-references every dependency against GitHub's Advisory Database



Viewable under a repository's Insights > Dependency graph tab



Enabled by default on public repositories; opt-in for private repositories

Exporting a Software Bill of Materials (SBOM)



A Software Bill of Materials lists every component in a project, direct and transitive



GitHub can export a repository's dependency graph as a standard SPDX-format SBOM



Increasingly required for compliance, procurement, and vendor security reviews



Exported from Insights > Dependency graph > Export SBOM



Regenerate and re-export periodically — an SBOM reflects a single point in time

Understanding Dependabot Alerts



Generated automatically when a dependency in the graph matches a known advisory



Repository maintainers are notified via the same notification system covered on Day 1



Each alert shows the vulnerable version range and the patched version, if one exists



Viewable under a repository's Security tab, or aggregated at the organization level



An alert closes automatically once the dependency is updated past the vulnerable range

Dependabot Alert Severity and CVSS Scoring



Critical

CVSS 9.0-10.0 — severe, often remotely exploitable with no user interaction required



High

CVSS 7.0-8.9 — significant impact, typically requiring some specific condition to exploit



Medium

CVSS 4.0-6.9 — real but limited impact, or requiring meaningful attacker access already



Low

CVSS 0.1-3.9 — minimal practical impact in most real-world configurations

Reviewing and Triaging a Dependabot Alert

- 1 Open the repository's Security tab, then click Dependabot alerts
- 2 Click into a specific alert to see the affected dependency, severity, and patched version
- 3 Review the Dependabot-generated pull request, if one has already been opened automatically
- 4 Check whether the vulnerable code path is actually reachable in your application
- 5 Merge the automated fix, or manually update if the automated PR doesn't apply cleanly
- 6 If genuinely not exploitable in your context, dismiss the alert with a documented reason

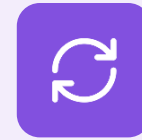
Security Updates vs. Version Updates



Security Updates

- Triggered automatically by a Dependabot alert
- Opens a pull request bumping only the vulnerable dependency
- Targets the minimum version that resolves the vulnerability
- On by default once Dependabot alerts are enabled

VS



Version Updates

- Runs on a schedule you define, independent of any vulnerability
- Opens pull requests keeping dependencies broadly current
- Configured explicitly via a `dependabot.yml` file
- Off by default — an opt-in, routine maintenance practice

Configuring Dependabot with dependabot.yml

```
.github/dependabot.yml

version: 2

updates:

  - package-ecosystem: "npm"

    directory: "/"

    schedule:

      interval: "weekly"

    open-pull-requests-limit: 5

  - package-ecosystem: "github-actions"
```

PREVIEW

What this configuration does

- package-ecosystem selects which dependency manager to monitor
- directory points at the manifest file's location
- schedule.interval sets how often to check for updates
- open-pull-requests-limit caps concurrent update PRs
- A second entry monitors GitHub Actions versions too

Dismissing a Dependabot Alert Responsibly



Dismissing does not fix the vulnerability — it only removes the alert from the active list



GitHub requires selecting a reason: false positive, not used, no fix available, or risk accepted



Dismissed alerts remain visible in a repository's dismissed alerts history for later review



Track who dismissed an alert and when — visible directly on the alert itself



A dismissed alert can be reopened later if circumstances change

Dependency Review on Pull Requests



Shows exactly which dependencies a pull request adds, removes, or updates



Flags any newly introduced dependency that carries a known vulnerability



Can also surface license information for every affected dependency



Can be configured as a required check, blocking merge on a vulnerable new dependency



Appears directly in the Files changed tab of any pull request that touches a manifest file

Introduction to Secret Scanning



Continuously scans a repository's full history for known credential patterns



Complements, and largely automates, the manual code-search technique from Day 2



Recognizes patterns from dozens of partner providers: cloud platforms, payment processors, and more



Both the repository and, where possible, the credential's provider are notified



Alerts are viewable under a repository's Security tab, alongside Dependabot alerts

Secret Scanning Partner Patterns



GitHub maintains partnerships with major cloud, payment, and API providers



Partner patterns are validated automatically — GitHub checks if a detected key is still active



A validated, active secret is treated with higher urgency than an unvalidated match



The list of supported partners and patterns grows continuously; no manual setup is required



Available on public repositories by default; opt-in for private repositories, like the dependency graph

Secret Scanning Push Protection



Blocks a push containing a detectable secret before it ever reaches GitHub



Runs at push time, not after the fact — prevention rather than detection



The developer sees the exact file, line, and secret type directly in their push output



Can be bypassed with a documented reason, for confirmed false positives



Can be enabled per repository or enforced organization-wide

Responding to a Push Protection Block

- 1 Read the block message to identify the exact file, line, and secret type flagged
- 2 Remove the secret from the file, replacing it with an environment variable or secret reference
- 3 If the secret was genuinely already exposed, revoke and rotate it with the issuing provider immediately
- 4 Amend the commit (if not yet pushed elsewhere) rather than adding a new commit that leaves the secret in history
- 5 Push again and confirm the block is now resolved
- 6 If it was truly a false positive, use the provided bypass option and document why

Custom Patterns for Secret Scanning



Define detection patterns for credentials specific to your own organization



Patterns are written as regular expressions matching your internal token format



Configured at the repository or organization level under Security settings



Test a custom pattern against existing repository content before enabling it broadly



Custom patterns can also participate in push protection, just like partner patterns

Introduction to Code Scanning and CodeQL



Analyzes a codebase's actual structure and logic, not just text patterns



CodeQL treats code itself as queryable data, similar to running a database query



Finds vulnerability classes like SQL injection, XSS, and unsafe deserialization



Runs as a GitHub Actions workflow, triggered on push or pull request



Results appear as annotated alerts, directly on the relevant line of code

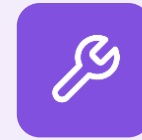
Default Setup vs. Advanced Setup for Code Scanning



Default Setup

- GitHub automatically configures and manages the CodeQL workflow for you
- Detects languages present and chooses sensible defaults automatically
- Enabled with a single click under Settings > Code security
- The right starting point for most repositories

VS



Advanced Setup

- You commit and fully control the CodeQL workflow YAML file yourself
- Customize query suites, scan schedule, and excluded paths precisely
- Required for languages or build steps default setup doesn't support well
- The right choice once a team needs fine-grained control

Anatomy of a CodeQL Analysis Workflow



name:

A human-readable name shown in the Security and Actions tabs



on:

Typically push, pull_request, and a weekly schedule for thorough coverage



init step

The github/codeql-action/init step selects languages and query suites



analyze step

Builds (if needed) and runs the actual CodeQL analysis



upload

Results are uploaded automatically and appear in the Security tab

CodeQL Query Packs and Language Support



CodeQL supports major languages: JavaScript/TypeScript, Python, Java, C#, Go, Ruby, C/C++



The default query pack balances thoroughness against false-positive rate



Extended query packs add more queries, catching more issues at the cost of more noise



Custom queries can be written for organization-specific patterns or frameworks



GitHub's security research team and the open-source community both contribute queries

Reading and Triaging Code Scanning Alerts



Each alert links directly to the specific line and shows the full data-flow path



For complex findings, CodeQL shows every intermediate step from source to vulnerable sink



Mark an alert as fixed by resolving it — closure is verified on the next scan



Dismiss with a reason: false positive, used in tests, or won't fix



Alerts can also appear as inline pull request comments, blocking review the same way CI checks do

Alert Severity Levels Across All Three Tools



Critical

Immediate action — remotely exploitable or actively confirmed as a live credential



High

Prompt action — significant impact, typically within days, not months



Medium / Low

Scheduled action — track and address as routine work, not an emergency

Security Advisories and Private Vulnerability Reporting



A security advisory is a structured record of a vulnerability and its fix, tied to a repository



Advisories start as private drafts, visible only to invited collaborators



Private vulnerability reporting lets outside researchers report issues directly and privately



Once published, an advisory can be assigned a CVE identifier for public tracking



Publishing triggers Dependabot alerts for every downstream repository depending on the fixed version

Publishing a Security Advisory

- 1 Open the repository's Security tab, then click Advisories, then New draft security advisory
- 2 Fill in a title, severity, and detailed description of the vulnerability
- 3 Identify the affected product and the version range considered vulnerable
- 4 Add temporary collaborators if outside help is needed while the fix is developed
- 5 Link the pull request that contains the actual fix once it's ready
- 6 Publish the advisory once the fix has been released, making it public

Writing a SECURITY.md Security Policy



A SECURITY.md file tells researchers and users exactly how to report a vulnerability



Placed at the repository root, or an organization's dedicated .github repository



Should specify a contact method and expected response time



Can point directly to private vulnerability reporting, if that feature is enabled



GitHub surfaces a Security policy link automatically once this file exists

The Organization Security Overview Dashboard



Aggregates Dependabot, secret scanning, and code scanning alerts across every repository



Filter by severity, alert type, team, or specific repository



Shows trend data: alert volume rising, falling, or steady over time



Available under the organization's own Security tab, not any single repository's



The natural daily or weekly starting point for a security-focused administrator

Enabling Security Features at Scale with Security Configurations



A security configuration is a reusable bundle of security settings



Define once — which features are on, which are required — then apply broadly



Apply to existing repositories individually, by group, or to all future new repositories



GitHub also provides a recommended default configuration as a starting point



Repositories not yet matching the configuration are clearly visible for follow-up

Managing and Prioritizing Alerts Across Many Repositories



Start from the organization-level security overview, not individual repositories



Sort by severity first — critical and high alerts across all repos before anything else



Route alerts to the right team using the same team structure from Day 2

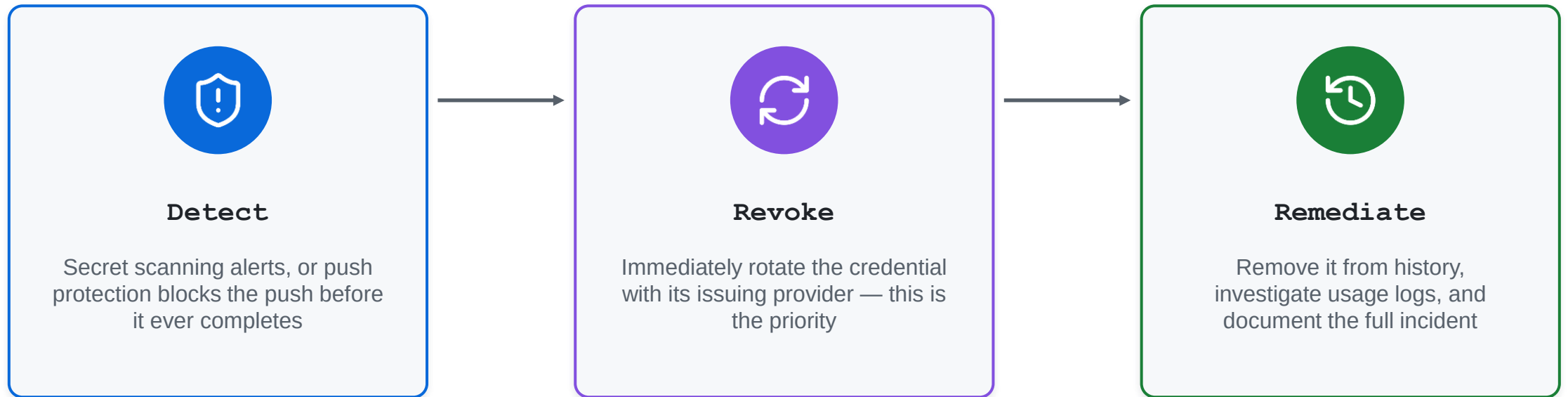


Track alert age — an unresolved critical alert open for weeks is its own signal



Review trend data periodically, not just the current snapshot

Incident Response: Handling a Leaked Secret



Security Roles and Responsibilities



Security Manager

An organization role with security-alert visibility across every repository, without full admin access



Repository Admin

Configures per-repository security settings and reviews alerts for repos they own



Organization Owner

Sets organization-wide security configurations and enforces baseline requirements

Compliance and Security Best Practices Recap



Enable the dependency graph, Dependabot alerts, and secret scanning by default on every repository



Enforce push protection organization-wide rather than leaving it optional per repository



Start code scanning with default setup; move to advanced setup only when a real limitation appears



Use security configurations so new repositories inherit a secure baseline automatically



Review the security overview dashboard on a regular, standing cadence

Module 8 Recap

- ✓ Dependency graph, Dependabot alerts, and both update types configured and triaged
- ✓ Secret scanning and push protection understood as detection versus prevention
- ✓ Code scanning and CodeQL set up, with alerts read and triaged confidently
- ✓ Security advisories and a SECURITY.md policy used for responsible disclosure
- ✓ Security managed at organization scale: configurations, the overview dashboard, and roles

CAPSTONE PROJECT

Capstone Group Project

Applying three days of GitHub administration to a realistic organization

Capstone Overview and Objectives



Teams of 3-4 set up a complete GitHub organization for a fictional company



Every deliverable draws on specific material from Days 1, 2, or 3 — nothing is invented from scratch



Teams present their setup and the reasoning behind their choices to the full class



Each team also reviews and gives structured feedback on one other team's setup



Evaluation weighs sound reasoning and trade-off awareness as heavily as correct configuration

Team Formation and Roles



Teams of 3-4 students, either self-selected or assigned by the instructor



Within each team, informally rotate who drives each major deliverable area



At least one team member should take primary ownership of the security configuration



Assign one team member to keep the documentation and rubric checklist current throughout



Decisions should be made as a team — the presentation will ask about ones a single member can't explain

Capstone Phases and Timeline

- 1 Phase 1: Form teams, choose a scenario, and draft an initial organization and team structure plan
- 2 Phase 2: Build the organization — accounts, teams, repositories, branch protection, and CI
- 3 Phase 3: Configure security — Dependabot, secret scanning, code scanning, and a security policy
- 4 Phase 4: Write governance documentation and complete the rubric self-checklist
- 5 Phase 5: Present your setup to the class and participate in peer review of another team

Capstone Requirement Areas



Access & Identity

Organization, teams, roles, and security baseline (2FA, SSO discussion)



Repos & CI/CD

Repository structure, branch protection, and at least one working Actions workflow



Security Configuration

Dependabot, secret scanning, code scanning, and a security policy



Documentation & Governance

Onboarding process, access rationale, and an incident response plan

Example Scenario Options



Seed-Stage Startup

12 engineers, one core product, moving fast with minimal existing process



Regulated Fintech

40 engineers, strict compliance requirements, and a mandatory audit trail



Open-Source Foundation

A small core team plus many external, unpaid contributors

Deliverable 1 — Organization and Team Structure



A created organization with a clear, documented team hierarchy



At least two organization owners — never a single point of failure



A defined base permission level, with reasoning for the choice



A documented position on 2FA and SSO enforcement, appropriate to the chosen scenario



A short written rationale explaining why the team structure fits the scenario

Deliverable 2 — Repository, Branch Protection, and CI



At least two repositories reflecting the fictional company's actual products or services



Branch protection on main requiring pull request review before merging



At least one working GitHub Actions workflow, triggered on pull_request



A completed pull request showing the full review-and-merge cycle from Day 2



A README for each repository, following Day 1's anatomy: purpose, install, usage, license

Deliverable 3 — Security Configuration



Dependency graph and Dependabot alerts enabled on every repository



Secret scanning enabled, with push protection turned on organization-wide



Code scanning enabled via default setup on at least one repository



A written SECURITY.md policy explaining how to report a vulnerability



A security configuration defined and applied, or a documented plan to create one

Deliverable 4 — Documentation and Governance Checklist



A written onboarding process a real new hire could actually follow



An adaptation of Day 2's five-category administration checklist to your own scenario



A stated plan for reviewing the audit log and security overview on a regular cadence



A short reflection on what you'd do differently with more time or a larger team



Clear attribution of which team member led which deliverable area

Deliverable 5 — Incident Response Tabletop Exercise



Choose one realistic incident scenario relevant to your fictional company



Walk through detection, response, and remediation using this module's framework



Identify specifically who on your team would be responsible for each response step



Document what evidence you'd gather and where you'd find it in GitHub's UI



Write a short summary as if reporting to leadership after the incident is resolved

Capstone Grading Rubric Breakdown



Access & Identity — 20%

Organization and team structure, roles, and documented security baseline decisions



Repos & CI/CD — 20%

Repository setup, branch protection, and a working automated workflow



Security Config — 25%

Today's Module 8 features genuinely configured, not merely described



Documentation — 20%

Onboarding process, checklist adaptation, and honest self-reflection



Presentation & Review — 15%

Clear reasoning when presenting, and thoughtful feedback during peer review

Presentation Format and Expectations



Each team presents for approximately 10 minutes, followed by 5 minutes of questions



Every team member should speak — the instructor may call on anyone directly



Live-demo your actual organization rather than only showing static slides or screenshots



Be ready to explain at least one deliberate trade-off you made and why



Questions from classmates and the instructor are a normal, expected part of every presentation

How to Submit Your Capstone Project

- 1 Confirm every deliverable is complete using the grading rubric as a final self-checklist
- 2 Ensure the instructor (or a shared reviewer account) has been added with appropriate access
- 3 Compile your written documentation into a single shared document or repository wiki
- 4 Submit the organization name and links to key repositories through the course's submission form
- 5 Confirm your presentation slot and that every team member is prepared to speak

Evaluating Another Team's Setup (Peer Review)



Review your assigned team's organization as if conducting a real security and access audit



Use the same rubric categories to structure your feedback consistently



Note at least two genuine strengths, specifically, not just generically positive comments



Note at least two concrete gaps or risks, with a suggested improvement for each



Deliver feedback respectfully and specifically — vague feedback helps no one improve

Common Capstone Pitfalls to Avoid



Configuring every setting without documenting why any of it was chosen



Treating Security Configuration as an afterthought squeezed in at the very end



One person doing most of the technical work while others only observe



Writing generic documentation that doesn't actually reference your specific scenario



Skipping the incident response deliverable because it feels less concrete than the others

Getting Help During the Capstone



Ask the instructor directly — clarifying questions during work time are expected and welcome



Revisit the relevant day's slides and lab guide before assuming something is unclear or missing



Compare notes with another team facing a similar technical question, without directly copying their work



Consult docs.github.com for authoritative detail on any specific feature



Flag early if your team is falling behind schedule, rather than waiting until presentation day

Capstone Project Recap

- ✓ Teams formed, a scenario chosen, and a team and permission structure planned before building
- ✓ All five deliverables completed: access, repositories/CI, security, documentation, and incident response
- ✓ Every choice documented with reasoning, not just configured and left unexplained
- ✓ Presentations delivered with every team member able to speak to the reasoning behind it
- ✓ Peer review completed with specific, constructive, actionable feedback for another team

Day 3 Key Takeaways

- ✓ Dependabot, secret scanning, and code scanning each address a distinct category of risk
- ✓ Prevention (push protection) is more powerful than detection alone, wherever it's available
- ✓ Security, like access control, must be managed at organization scale, not repository by repository
- ✓ Responsible disclosure — advisories and a security policy — is a process, not just a technical control
- ✓ The capstone proved these tools mean little without documented reasoning behind how they're used

Day 3 Terminology Cheat Sheet



Dependabot

GitHub's tool for finding and fixing vulnerable or outdated dependencies



Secret Scanning

Continuous detection of credentials committed to a repository



Push Protection

Blocks a push containing a detectable secret before it completes



CodeQL

GitHub's code analysis engine, treating code as queryable data



Security Advisory

A structured record for privately fixing, then disclosing, a vulnerability



SBOM

Software Bill of Materials — a full export of a project's dependencies



Security Configuration

A reusable bundle of security settings applied across repositories



Security Manager

An org role with security visibility, without full administrative access

Common Pitfalls & Best Practices



Enable Dependabot, secret scanning, and push protection as an organization-wide default, not an opt-in



Start code scanning with default setup — move to advanced setup only once truly needed



Write a SECURITY.md before you need one, not after your first real vulnerability report



Use security configurations so every new repository inherits a secure baseline automatically



Document the reasoning behind every access and security decision — not just the setting itself

Day 3 Lab Overview (Part 1 of 2)

1

Exploring the Dependency Graph and Exporting an SBOM

2

Triaging a Dependabot Alert and Applying a Security Update

3

Configuring Dependabot Version Updates with dependabot.yml

4

Reviewing Dependency Changes on a Pull Request

5

Enabling Secret Scanning and Investigating an Alert

6

Testing Secret Scanning Push Protection

7

Creating a Custom Secret Scanning Pattern

8

Enabling Code Scanning with Default Setup

Day 3 Lab Overview (Part 2 of 2)

9

Configuring Code Scanning with a Custom CodeQL Workflow

10

Triaging and Resolving a Code Scanning Alert

11

Publishing a Security Advisory

12

Writing a Security Policy (SECURITY.md)

13

Enabling Security Configurations Across Multiple Repositories

14

Reviewing the Organization Security Overview Dashboard

15

Running a Tabletop Incident Response Exercise

Resources & Further Reading



GitHub Docs — docs.github.com/code-security, covering every feature from today in full depth



GitHub Advisory Database — github.com/advisories, the source behind every Dependabot alert



CodeQL Documentation and Query Help — codeql.github.com



GitHub Skills — skills.github.com, including a dedicated security-focused course



This course's full three-day lab guide — the most complete reference to take with you

CONGRATULATIONS

Course Complete — Continuing Your GitHub Administration Journey



You've completed all three days: foundations, collaboration, and security — congratulations



GitHub Skills (skills.github.com) offers free, ongoing practice beyond this course



Revisit your Day 1-3 glossaries and checklists as a working reference in your own role



Apply your capstone's structure — teams, security configuration, documentation — to a real organization